WebSocket Lab - Sample Code Guide

Purpose: This guide contains detailed sample code with explanations to help you understand WebSocket development with Go before attempting the lab tasks.

How to Use This Guide:

1. Read through each sample carefully
2. Run the sample code on your machine
3. Study the explanations to understand how it works
4. Test the services using browser DevTools or the Go client
5. Then attempt the corresponding tasks in the WebSocket Lab Tasks file

Prerequisites: You should have read the **WebSocket Tutorial** document first. This guide reinforces those concepts with hands-on, runnable examples and detailed explanations.

Table of Contents

1. Part 1: WebSocket Basics — Upgrader and Echo
2. Part 2: JSON Message Protocol
3. Part 3: The Hub Pattern — Broadcasting
4. Part 4: Client Struct — Read Pump and Write Pump
5. Part 5: Go Client for WebSocket
6. Part 6: Connection Management and Error Handling
7. Part 7: Testing WebSocket Services
8. Common Mistakes to Avoid
9. Quick Reference

Part 1: WebSocket Basics — Upgrader and Echo

📖 Sample Code: Your First WebSocket Server

The simplest WebSocket server receives a message and sends it back — an "echo" server. This teaches the fundamental connection lifecycle: upgrade, read, write, close.

Step 1 — Setup the project:

```
mkdir ws-sample
cd ws-sample
go mod init ws-sample
```

Step 2 — Install the WebSocket library:

```
go get github.com/gorilla/websocket
```

Step 3 — Create `server/main.go`:

```go
package main

import (
	"log"
	"net/http"

	"github.com/gorilla/websocket"
)

// upgrader converts HTTP connections to WebSocket connections.
var upgrader = websocket.Upgrader{
	ReadBufferSize:  1024,
	WriteBufferSize: 1024,
	CheckOrigin: func(r *http.Request) bool {
		return true // Allow all origins (development only!)
	},
}

// handleWebSocket upgrades the HTTP connection and echoes messages back.
func handleWebSocket(w http.ResponseWriter, r *http.Request) {
	// Step 1: Upgrade HTTP → WebSocket
	conn, err := upgrader.Upgrade(w, r, nil)
	if err != nil {
		log.Println("upgrade error:", err)
		return
	}
	defer conn.Close()

	log.Printf("client connected: %s", conn.RemoteAddr())

	// Step 2: Read loop — continuously read messages and echo them back
	for {
		messageType, message, err := conn.ReadMessage()
		if err != nil {
			log.Printf("client disconnected: %s (%v)", conn.RemoteAddr(),
			break
		}

		log.Printf("received: %s", message)

		// Step 3: Echo the message back
		if err := conn.WriteMessage(messageType, message); err != nil {
			log.Println("write error:", err)
			break
		}
	}
}

func main() {
	http.HandleFunc("/ws", handleWebSocket)
	http.Handle("/", http.FileServer(http.Dir("static")))

	log.Println("Echo server listening on :8080")
	log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Step 4 — Create `static/index.html`:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>WebSocket Echo Test</title>
    <style>
        body { font-family: monospace; max-width: 600px; margin: 40px auto; }
        #log { border: 1px solid #ccc; padding: 10px; height: 300px; overflow-y: aut
               background: #f9f9f9; margin-bottom: 10px; }
        .sent { color: blue; }
        .received { color: green; }
        .system { color: gray; font-style: italic; }
        input { width: 400px; padding: 5px; }
        button { padding: 5px 15px; }
    </style>
</head>
<body>
    <h2>WebSocket Echo Test</h2>
    <div id="log"></div>
    <input type="text" id="msg" placeholder="Type a message..." />
    <button onclick="send()">Send</button>
    <button onclick="ws.close()">Disconnect</button>

    <script>
        const logDiv = document.getElementById('log');
        const msgInput = document.getElementById('msg');

        function appendLog(text, cls) {
            const d = document.createElement('div');
            d.textContent = text;
            d.className = cls;
            logDiv.appendChild(d);
            logDiv.scrollTop = logDiv.scrollHeight;
        }

        // Open WebSocket connection
        const ws = new WebSocket('ws://localhost:8080/ws');

        ws.onopen = () => appendLog('Connected to server', 'system');
        ws.onmessage = (e) => appendLog('← ' + e.data, 'received');
        ws.onclose = (e) => appendLog('Disconnected (code: ' + e.code + ')', 'system
        ws.onerror = () => appendLog('Error occurred', 'system');

        function send() {
            const text = msgInput.value;
            if (!text || ws.readyState !== WebSocket.OPEN) return;
            ws.send(text);
            appendLog('→ ' + text, 'sent');
            msgInput.value = '';
        }

        msgInput.addEventListener('keypress', (e) => { if (e.key === 'Enter') send()
    </script>
</body>
</html>
```

Project structure:

```
ws-sample/
├── go.mod
├── server/
│   └── main.go         ← echo server
└── static/
    └── index.html      ← browser test client
```

Run:

```
go run ./server
```

Open `http://localhost:8080` in your browser. Type a message, press Send — you'll see it echoed back.

📝 Detailed Code Explanation:

Understanding the Upgrader

```go
var upgrader = websocket.Upgrader{
    ReadBufferSize:  1024,
    WriteBufferSize: 1024,
    CheckOrigin: func(r *http.Request) bool {
        return true
    },
}
```

- The `Upgrader` is responsible for converting an HTTP connection into a WebSocket connection
- `ReadBufferSize` / `WriteBufferSize` set the I/O buffer sizes (1024 bytes is fine for small messages)
- `CheckOrigin` is a security function that decides whether to accept the connection based on the HTTP `Origin` header
- Setting `CheckOrigin` to always return `true` disables origin checking — **only for development**. In production, verify the origin matches your domain

Without `CheckOrigin`:

```go
// ❌ Default Upgrader rejects cross-origin requests
var upgrader = websocket.Upgrader{}
// Browser connections from http://localhost:8080 will be rejected
// because the default CheckOrigin requires the Origin header to match the Host
```

With `CheckOrigin`:

```go
// ✅ Allows connections from any origin (development)
var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool { return true },
}
```

Understanding `upgrader.Upgrade()`

```go
conn, err := upgrader.Upgrade(w, r, nil)
```

- Takes the HTTP `ResponseWriter` and `Request` — these are consumed by the upgrade
- The third parameter (`nil`) is optional response headers to include in the handshake
- Returns a `*websocket.Conn` — your WebSocket connection object
- After `Upgrade`, you **cannot** use `w` or `r` anymore — the connection has changed protocols
- If the upgrade fails (e.g., the client didn't send proper WebSocket headers), `Upgrade` writes an HTTP error response automatically

Understanding the Read Loop

```go
for {
    messageType, message, err := conn.ReadMessage()
    if err != nil {
        break
    }
    // handle message...
}
```

- `ReadMessage()` **blocks** until a message arrives or the connection closes
- Returns three values:
  - `messageType`: `websocket.TextMessage` (1) or `websocket.BinaryMessage` (2)
  - `message`: the payload as `[]byte`
  - `err`: non-nil when the connection is closed or broken
- The read loop is the **heartbeat** of your WebSocket handler — without it, the connection closes immediately
- When `ReadMessage` returns an error, the connection is dead — exit the loop and clean up

`ReadMessage()` returns	Meaning	Action
`(type, data, nil)`	Message received	Process the message
`(0, nil, error)`	Connection closed or broken	Break the loop, clean up

Understanding `conn.WriteMessage()`

```go
err := conn.WriteMessage(messageType, message)
```

- Sends a message to the connected client
- `messageType` should match what the client expects: `websocket.TextMessage` for text/JSON, `websocket.BinaryMessage` for binary data
- Returns an error if the connection is broken
- **Not safe for concurrent use** — if two goroutines call `WriteMessage` at the same time, you get corrupted frames or a panic

Understanding `defer conn.Close()`

```go
defer conn.Close()
```

- Ensures the WebSocket connection is properly closed when the handler function exits
- Sends a close frame to the client if the connection is still open
- Without this, you'll leak TCP connections

Understanding the JavaScript WebSocket API

```javascript
const ws = new WebSocket('ws://localhost:8080/ws');
```

Property/Method	Purpose
`new WebSocket(url)`	Opens a WebSocket connection
`ws.send(data)`	Sends a text or binary message
`ws.close()`	Initiates the closing handshake
`ws.readyState`	`0` =CONNECTING, `1` =OPEN, `2` =CLOSING, `3` =CLOSED
`ws.onopen`	Callback: connection established
`ws.onmessage`	Callback: message received (data in `event.data`)
`ws.onclose`	Callback: connection closed (code in `event.code`)
`ws.onerror`	Callback: error occurred

Key Concepts from Part 1

1. **Upgrade handshake**: HTTP → WebSocket via `upgrader.Upgrade()`
2. **Read loop**: the `for { ReadMessage() }` pattern keeps the connection alive
3. **Connection lifecycle**: upgrade → read/write loop → close
4. **Thread safety**: `WriteMessage` is NOT concurrent-safe — one writer at a time
5. **`CheckOrigin`**: required for cross-origin browser connections during development

---

Part 2: JSON Message Protocol

📖 Sample Code: Structured Messages with JSON

Sending raw text strings works for simple echo, but real applications need structured messages — who sent it, what type it is, when it was sent. Since WebSocket has no built-in schema (unlike gRPC's protobuf), you define your own message format using JSON.

Update `server/main.go` to work with JSON messages:

```go
package main

import (
	"encoding/json"
	"log"
	"net/http"
	"time"

	"github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
	ReadBufferSize:  1024,
	WriteBufferSize: 1024,
	CheckOrigin: func(r *http.Request) bool { return true },
}

// Message defines the JSON structure for all WebSocket communication.
type Message struct {
	Type      string `json:"type"`
	Sender    string `json:"sender"`
	Content   string `json:"content"`
	Timestamp string `json:"timestamp,omitempty"`
}

func handleWebSocket(w http.ResponseWriter, r *http.Request) {
	conn, err := upgrader.Upgrade(w, r, nil)
	if err != nil {
		log.Println("upgrade error:", err)
		return
	}
	defer conn.Close()

	log.Printf("client connected: %s", conn.RemoteAddr())

	for {
		// Read raw bytes from WebSocket
		_, rawMessage, err := conn.ReadMessage()
		if err != nil {
			log.Printf("client disconnected: %v", err)
			break
		}

		// Parse JSON into Message struct
		var msg Message
		if err := json.Unmarshal(rawMessage, &msg); err != nil {
			log.Printf("invalid JSON: %v", err)

			// Send an error message back to the client
			errMsg := Message{
				Type:    "error",
				Content: "invalid JSON format",
			}
			errBytes, _ := json.Marshal(errMsg)
			conn.WriteMessage(websocket.TextMessage, errBytes)
			continue
		}

		// Validate required fields
		if msg.Type == "" || msg.Content == "" {
			errMsg := Message{
				Type:    "error",
				Content: "missing required fields: type and content",
			}
			errBytes, _ := json.Marshal(errMsg)
			conn.WriteMessage(websocket.TextMessage, errBytes)
			continue
		}

		log.Printf("[%s] %s: %s", msg.Type, msg.Sender, msg.Content)

		// Route based on message type
		switch msg.Type {
		case "chat":
			// Add server timestamp and echo back
			msg.Timestamp = time.Now().Format(time.RFC3339)
			response, _ := json.Marshal(msg)
			conn.WriteMessage(websocket.TextMessage, response)

		default:
			errMsg := Message{
				Type:    "error",
				Content: "unknown message type: " + msg.Type,
			}
			errBytes, _ := json.Marshal(errMsg)
			conn.WriteMessage(websocket.TextMessage, errBytes)
		}
	}
}

func main() {
	http.HandleFunc("/ws", handleWebSocket)
	http.Handle("/", http.FileServer(http.Dir("static")))

	log.Println("JSON Echo server listening on :8080")
	log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Update `static/index.html` to send/receive JSON:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>WebSocket JSON Test</title>
    <style>
        body { font-family: monospace; max-width: 600px; margin: 40px auto; }
        #log { border: 1px solid #ccc; padding: 10px; height: 300px; overflow-y: aut
               background: #f9f9f9; margin-bottom: 10px; }
        .sent { color: blue; }
        .received { color: green; }
        .error { color: red; }
        .system { color: gray; font-style: italic; }
        input { width: 300px; padding: 5px; }
        button { padding: 5px 15px; }
    </style>
</head>
<body>
    <h2>WebSocket JSON Test</h2>
    <div id="log"></div>
    <input type="text" id="name" placeholder="Your name" value="Alice" style="width:
    <input type="text" id="msg" placeholder="Type a message..." />
    <button onclick="send()">Send</button>

    <script>
        const logDiv = document.getElementById('log');
        const nameInput = document.getElementById('name');
        const msgInput = document.getElementById('msg');

        function appendLog(text, cls) {
            const d = document.createElement('div');
            d.textContent = text;
            d.className = cls;
            logDiv.appendChild(d);
            logDiv.scrollTop = logDiv.scrollHeight;
        }

        const ws = new WebSocket('ws://localhost:8080/ws');

        ws.onopen = () => appendLog('Connected', 'system');

        ws.onmessage = (e) => {
            const msg = JSON.parse(e.data);
            if (msg.type === 'error') {
                appendLog('Error: ' + msg.content, 'error');
            } else {
                appendLog(`← [${msg.type}] ${msg.sender}: ${msg.content} (${msg.time
            }
        };

        ws.onclose = () => appendLog('Disconnected', 'system');

        function send() {
            const text = msgInput.value;
            if (!text || ws.readyState !== WebSocket.OPEN) return;

            // Build a JSON message
            const msg = {
                type: 'chat',
                sender: nameInput.value || 'Anonymous',
                content: text
            };

            ws.send(JSON.stringify(msg));
            appendLog(`→ [${msg.type}] ${msg.sender}: ${msg.content}`, 'sent');
            msgInput.value = '';
        }

        msgInput.addEventListener('keypress', (e) => { if (e.key === 'Enter') send()
    </script>
</body>
</html>
```

Run and test:

```
go run ./server
# Open http://localhost:8080 in browser
# Type a message and send — you'll see JSON echoed back with a timestamp
```

📝 Detailed Code Explanation:

Understanding the Message Struct

```go
type Message struct {
    Type      string `json:"type"`
    Sender    string `json:"sender"`
    Content   string `json:"content"`
    Timestamp string `json:"timestamp,omitempty"`
}
```

- This struct defines your **message protocol** — the contract between client and server
- `json:"type"` tags map Go field names to JSON keys
- `omitempty` on `Timestamp` means the field is excluded from JSON if empty (client messages don't have timestamps — the server adds them)
- The `Type` field is critical: it tells the server how to handle the message

Understanding `json.Unmarshal`

```go
var msg Message
if err := json.Unmarshal(rawMessage, &msg); err != nil {
    // Invalid JSON — send error back
}
```

- `json.Unmarshal(data, &target)` parses JSON bytes into a Go struct
- Returns an error if the JSON is malformed (missing braces, invalid syntax)
- Unknown JSON fields are silently ignored by default
- Missing struct fields get their Go zero values (`""` for strings, `0` for numbers)

Understanding Message Routing

```go
switch msg.Type {
case "chat":
    // Handle chat messages
default:
    // Unknown type — send error
}
```

- The `switch msg.Type` pattern is how you route messages to different handlers
- This is the WebSocket equivalent of URL routing in REST — but the "routes" are message types, not URL paths
- Adding new features means adding new `case` branches

Understanding `json.Marshal`

```go
response, _ := json.Marshal(msg)
conn.WriteMessage(websocket.TextMessage, response)
```

- `json.Marshal(value)` converts a Go struct to JSON bytes
- The result is sent as a `TextMessage` (not `BinaryMessage`) because JSON is text
- On the JavaScript side, `JSON.parse(event.data)` converts it back to a JS object

Key JSON Protocol Concepts

1. **Define a struct**: your message format is a Go struct with JSON tags
2. **Unmarshal incoming**: parse raw bytes into the struct for validation and routing
3. **Route on `type`**: use a `switch` to handle different message types
4. **Marshal outgoing**: convert structs back to JSON bytes before sending
5. **Validate input**: always check required fields — WebSocket has no schema enforcement

---

Part 3: The Hub Pattern — Broadcasting

📖 Sample Code: Sending Messages to All Connected Clients

An echo server handles each client independently. A chat server needs to **broadcast** — when one client sends a message, all clients receive it. The **Hub** is a central goroutine that manages all connected clients and dispatches messages.

Create `hub.go` in the project root (not in `server/`):

```go
package main

import "log"

// Hub maintains the set of active clients and broadcasts messages to them.
type Hub struct {
	clients    map[*Client]bool // All connected clients
	broadcast  chan []byte      // Messages to broadcast to all clients
	register   chan *Client     // Clients requesting to connect
	unregister chan *Client     // Clients requesting to disconnect
}

func newHub() *Hub {
	return &Hub{
		clients:    make(map[*Client]bool),
		broadcast:  make(chan []byte),
		register:   make(chan *Client),
		unregister: make(chan *Client),
	}
}

// run is the Hub's main event loop. Must be launched as a goroutine.
func (h *Hub) run() {
	for {
		select {
		case client := <-h.register:
			h.clients[client] = true
			log.Printf("client registered (%d total)", len(h.clients))

		case client := <-h.unregister:
			if _, ok := h.clients[client]; ok {
				delete(h.clients, client)
				close(client.send)
				log.Printf("client unregistered (%d remaining)", len(
			}

		case message := <-h.broadcast:
			for client := range h.clients {
				select {
				case client.send <- message:
					// Message queued to this client
				default:
					// Client's send buffer is full — disconnect
					close(client.send)
					delete(h.clients, client)
				}
			}
		}
	}
}
```

📝 Detailed Code Explanation:

Understanding the Hub's Channels

```go
type Hub struct {
    clients    map[*Client]bool
    broadcast  chan []byte
    register   chan *Client
    unregister chan *Client
}
```

Field	Type	Purpose
`clients`	`map[*Client]bool`	The set of currently connected clients. Using a map as a set (value is always `true`).
`broadcast`	`chan []byte`	Messages arrive here from any client's read goroutine. The Hub sends them to everyone.
`register`	`chan *Client`	When a new WebSocket connection is established, the Client is sent here.
`unregister`	`chan *Client`	When a client disconnects, it sends itself here for cleanup.

Understanding the `select` Event Loop

```go
func (h *Hub) run() {
    for {
        select {
        case client := <-h.register:
            // ...
        case client := <-h.unregister:
            // ...
        case message := <-h.broadcast:
```

```
		case message := <-h.broadcast:
			// ...
		}
	}
}
```

- The `select` statement blocks until one of the channels has data
- Only **one case** executes per iteration — this means the Hub processes events one at a time
- This is a single-goroutine event loop — no locks needed because only the Hub goroutine reads/writes `h.clients`
- This pattern is idiomatic Go: use channels for coordination instead of mutexes

Understanding the Broadcast Logic

```
case message := <-h.broadcast:
    for client := range h.clients {
        select {
        case client.send <- message:
            // Message queued
        default:
            // Buffer full — disconnect
            close(client.send)
            delete(h.clients, client)
        }
    }
```

- Iterates over every connected client and tries to push the message into their `send` channel
- The inner `select` with a `default` case is a **non-blocking send**: if the channel is full, it skips immediately
- A full `send` channel means the client can't keep up (network is slow, client is stuck) — so the Hub disconnects it
- This prevents one slow client from blocking all others

Why Single Goroutine?

The Hub manages shared state (`h.clients` map) in a single goroutine. This eliminates the need for `sync.Mutex`:

```
// WITHOUT Hub (mutex approach — works but error-prone):
var mu sync.Mutex
var clients map[*Client]bool

mu.Lock()
clients[newClient] = true
mu.Unlock()

// WITH Hub (channel approach — idiomatic Go):
hub.register <- newClient
// The Hub goroutine handles the map update — no locks needed
```

Key Hub Concepts

1. **Single goroutine**: the Hub runs in one goroutine — no locking needed
2. **Channel-based communication**: register, unregister, and broadcast via channels
3. **Non-blocking sends**: the `default` case prevents slow clients from blocking
4. **Cleanup on full buffer**: clients that can't keep up are disconnected automatically

---

Part 4: Client Struct — Read Pump and Write Pump

📖 Sample Code: The Client with Dedicated Read/Write Goroutines

Each WebSocket connection gets a **Client** struct with two goroutines: one for reading (`readPump`) and one for writing (`writePump`). This ensures only one goroutine writes to the connection at a time.

Create `client.go`:

```
package main

import (
	"log"
	"time"

	"github.com/gorilla/websocket"
)

const (
	writeWait      = 10 * time.Second    // Max time to write a message
	pongWait       = 60 * time.Second    // Max time to wait for a pong
	pingPeriod     = (pongWait * 9) / 10 // Send pings at this interval
	maxMessageSize = 512                 // Max message size in bytes
)

// Client represents a single WebSocket connection.
type Client struct {
	hub  *Hub
	conn *websocket.Conn
	send chan []byte // Buffered channel of outbound messages
}

// readPump reads messages from the WebSocket and forwards them to the Hub.
func (c *Client) readPump() {
	defer func() {
		c.hub.unregister <- c
		c.conn.Close()
	}()

	c.conn.SetReadLimit(maxMessageSize)
	c.conn.SetReadDeadline(time.Now().Add(pongWait))
	c.conn.SetPongHandler(func(string) error {
		c.conn.SetReadDeadline(time.Now().Add(pongWait))
		return nil
	})

	for {
		_, message, err := c.conn.ReadMessage()
		if err != nil {
			if websocket.IsUnexpectedCloseError(err,
				websocket.CloseGoingAway, websocket.CloseNormalClosur
				log.Printf("unexpected close: %v", err)
			}
			break
		}
		c.hub.broadcast <- message
	}
}

// writePump sends messages from the Hub to the WebSocket connection.
func (c *Client) writePump() {
	ticker := time.NewTicker(pingPeriod)
	defer func() {
		ticker.Stop()
		c.conn.Close()
	}()

	for {
		select {
		case message, ok := <-c.send:
			c.conn.SetWriteDeadline(time.Now().Add(writeWait))
			if !ok {
				// Hub closed our send channel — send close frame
				c.conn.WriteMessage(websocket.CloseMessage, []byte{})
				return
			}
			if err := c.conn.WriteMessage(websocket.TextMessage, message)
				return
			}

		case <-ticker.C:
			// Send periodic ping to detect dead connections
			c.conn.SetWriteDeadline(time.Now().Add(writeWait))
			if err := c.conn.WriteMessage(websocket.PingMessage, nil); er
				return
			}
		}
	}
}
```

Update `main.go` to wire everything together:

```
package main

import (
	"log"
	"net/http"

	"github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
	ReadBufferSize:  1024,
	WriteBufferSize: 1024,
	CheckOrigin: func(r *http.Request) bool { return true },
}

func serveWs(hub *Hub, w http.ResponseWriter, r *http.Request) {
	conn, err := upgrader.Upgrade(w, r, nil)
	if err != nil {
		log.Println("upgrade error:", err)
		return
	}

	client := &Client{
		hub:  hub,
		conn: conn,
		send: make(chan []byte, 256),
	}
	client.hub.register <- client

	// Launch both pumps — they take over ownership of the connection
	go client.writePump()
	go client.readPump()
}

func main() {
	hub := newHub()
	go hub.run()

	http.HandleFunc("/ws", func(w http.ResponseWriter, r *http.Request) {
		serveWs(hub, w, r)
	})
	http.Handle("/", http.FileServer(http.Dir("static")))

	log.Println("Broadcast server listening on :8080")
	log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Note: The code files (`hub.go`, `client.go`, `main.go`) are in the project root, not in a `server/` subdirectory. Run with `go run .` from the project root.

Run and test:

```
go run .
# Open http://localhost:8080 in TWO browser tabs
# Type in one tab — the message appears in BOTH tabs
```

📝 Detailed Code Explanation:

Understanding the readPump

```
func (c *Client) readPump() {
    defer func() {
        c.hub.unregister <- c
        c.conn.Close()
    }()
    // ...
    for {
        _, message, err := c.conn.ReadMessage()
        if err != nil { break }
        c.hub.broadcast <- message
    }
}
```

The `readPump` does three things:

1. **Reads** messages from the WebSocket connection
2. **Forwards** each message to `hub.broadcast` (so it reaches all clients)
3. **Cleans up** on disconnect: unregisters from the Hub and closes the connection

The `defer` block at the top ensures cleanup **always happens**, even if `ReadMessage` returns an unexpected error.

Understanding the writePump

```
func (c *Client) writePump() {
    ticker := time.NewTicker(pingPeriod)
    defer func() {
        ticker.Stop()
        c.conn.Close()
    }()

    for {
        select {
        case message, ok := <-c.send:
            if !ok {
                conn.WriteMessage(websocket.CloseMessage, []byte{})
                return
            }
            conn.WriteMessage(websocket.TextMessage, message)
        case <-ticker.C:
            conn.WriteMessage(websocket.PingMessage, nil)
        }
    }
}
```

The `writePump` has two jobs:

1. **Drain the `send` channel**: when the Hub pushes a message into `client.send`, the writePump picks it up and writes it to the WebSocket
2. **Send periodic pings**: the `ticker` fires every `pingPeriod` (54 seconds) to keep the connection alive

The `ok` variable from `<-c.send`:

- `ok == true`: a message was received — write it to the WebSocket
- `ok == false`: the channel was **closed** by the Hub — this means the Hub disconnected us. Send a close frame and exit.

Understanding Ping/Pong Keep-Alive

```
// readPump: expect pongs
c.conn.SetReadDeadline(time.Now().Add(pongWait))
c.conn.SetPongHandler(func(string) error {
    c.conn.SetReadDeadline(time.Now().Add(pongWait))
    return nil
})

// writePump: send pings
case <-ticker.C:
    conn.WriteMessage(websocket.PingMessage, nil)
```

The flow:

1. `writePump` sends a **Ping** every 54 seconds
2. The browser automatically responds with a **Pong** (the WebSocket spec requires this)
3. `readPump`'s `PongHandler` resets the read deadline when a Pong arrives
4. If no Pong arrives within 60 seconds (`pongWait`), `ReadMessage` returns a deadline error → the client is considered dead

Without ping/pong, a client whose network silently drops would stay in the Hub's client list forever, receiving broadcast attempts that go nowhere.

Understanding `IsUnexpectedCloseError`

```
if websocket.IsUnexpectedCloseError(err,
    websocket.CloseGoingAway, websocket.CloseNormalClosure) {
    log.Printf("unexpected close: %v", err)
}
```

- `CloseGoingAway` (1001): browser tab closed, page navigated away — **normal**
- `CloseNormalClosure` (1000): client called `ws.close()` — **normal**
- Any other close error is **unexpected** (network drop, protocol violation) — worth logging

This prevents your logs from being cluttered with normal disconnection messages.

Understanding the Connection Handoff

```
func serveWs(hub *Hub, w http.ResponseWriter, r *http.Request) {
    conn, _ := upgrader.Upgrade(w, r, nil)

    client := &Client{hub: hub, conn: conn, send: make(chan []byte, 256)}
    client.hub.register <- client

    go client.writePump()
    go client.readPump()
}
```

After `Upgrade`, the HTTP handler creates a Client and launches two goroutines. The handler then **returns** — the goroutines take full ownership of the connection from this point. This is critical: the HTTP handler's job is done after the upgrade. The connection's lifetime is now managed by the pumps.

The `send` channel has a buffer of 256 messages. If the Hub tries to send more than 256 messages before the writePump can process them, the Hub's non-blocking send will trigger the `default` case and disconnect the client.

Two-Goroutine Architecture

```
Per WebSocket connection:

  WebSocket ——ReadMessage()——▸ readPump goroutine ——▸ hub.broadcast
                                                            │
                                                            ▼
                                                        Hub.run()
                                                            │
                                                       client.send
                                                            │
                                                            ▼
  WebSocket ◂——WriteMessage()—— writePump goroutine ◂———┘
```

- **readPump**: reads from WebSocket → pushes to Hub
- **writePump**: reads from Hub → writes to WebSocket
- Neither goroutine touches the other's WebSocket direction
- Only `writePump` ever writes to the WebSocket — no concurrency issues

Key Client Concepts

1. **Two goroutines per client**: readPump reads, writePump writes — never mixed
2. **Channel-based handoff**: messages flow through channels, not direct function calls
3. **Ping/pong heartbeat**: detects dead connections that stopped responding
4. **Buffered send channel**: absorbs bursts; full buffer → client disconnected
5. **Cleanup in `defer`**: both pumps clean up on exit, preventing resource leaks

---

Part 5: Go Client for WebSocket

📖 Sample Code: Connecting from a Go Program

Besides browser clients, you can connect to WebSocket servers from Go programs. This is useful for CLI tools, testing, and service-to-service communication.

Create `go-client/main.go`:

```
package main

import (
	"bufio"
	"fmt"
	"log"
	"os"
	"os/signal"

	"github.com/gorilla/websocket"
)

func main() {
	// Connect to the WebSocket server
	conn, _, err := websocket.DefaultDialer.Dial("ws://localhost:8080/ws", nil)
	if err != nil {
		log.Fatal("dial error:", err)
	}
	defer conn.Close()

	log.Println("connected to ws://localhost:8080/ws")

	// Handle Ctrl+C
	interrupt := make(chan os.Signal, 1)
	signal.Notify(interrupt, os.Interrupt)

	// Goroutine to receive messages from server
	done := make(chan struct{})
	go func() {
		defer close(done)
		for {
			_, message, err := conn.ReadMessage()
			if err != nil {
				log.Println("disconnected:", err)
				return
			}
			fmt.Printf("  ← %s\n", message)
		}
	}()

	// Main goroutine: read user input and send
	scanner := bufio.NewScanner(os.Stdin)
	fmt.Println("Type messages and press Enter. Ctrl+C to quit.")

	for {
		select {
		case <-done:
			return
		case <-interrupt:
			log.Println("shutting down...")
			conn.WriteMessage(
				websocket.CloseMessage,
				websocket.FormatCloseMessage(websocket.CloseNormalClo
			)
			return
		default:
			if scanner.Scan() {
				text := scanner.Text()
				if err := conn.WriteMessage(websocket.TextMessage, []
					log.Println("write error:", err)
					return
				}
			}
		}
	}
}
```

📝 Detailed Code Explanation:

Understanding `websocket.DefaultDialer.Dial`

```
conn, _, err := websocket.DefaultDialer.Dial("ws://localhost:8080/ws", nil)
```

- `DefaultDialer` is a pre-configured dialer with sensible defaults
- `Dial(url, header)` performs the WebSocket handshake — the Go equivalent of `new WebSocket(url)` in JavaScript
- Returns `(*websocket.Conn, *http.Response, error)` — we ignore the HTTP response (the `_`)
- The second parameter (`nil`) is optional HTTP headers to send during the handshake (useful for authentication tokens)

Understanding Graceful Shutdown

```
conn.WriteMessage(
    websocket.CloseMessage,
    websocket.FormatCloseMessage(websocket.CloseNormalClosure, "bye"),
)
```

- `websocket.CloseMessage` is the frame type for close frames
- `FormatCloseMessage(code, reason)` creates a properly formatted close payload
- `CloseNormalClosure` (1000) = clean shutdown
- This tells the server "I'm leaving normally" — the server will see a clean close instead of an abrupt disconnect

Understanding the Two-Goroutine Client Pattern

```
// Goroutine 1: reads from server
go func() {
    for {
        _, msg, err := conn.ReadMessage()
        // ...
    }
}()

// Goroutine 2 (main): reads from stdin and writes to server
for {
    if scanner.Scan() {
        conn.WriteMessage(websocket.TextMessage, []byte(text))
    }
}
```

- The **reader goroutine** continuously receives server messages and prints them
- The **main goroutine** reads user input from stdin and sends messages
- This mirrors the server's readPump/writePump pattern
- `ReadMessage` blocks, so it must be in its own goroutine to not block stdin reading

---

Part 6: Connection Management and Error Handling

📖 Sample Code: Production-Quality Connection Handling

Read/Write Deadlines

```
// Set a deadline for the next write operation
c.conn.SetWriteDeadline(time.Now().Add(writeWait))
if err := c.conn.WriteMessage(websocket.TextMessage, data); err != nil {
    // Write timed out or connection broken
    return
}
```

- `SetWriteDeadline` sets a timeout for the **next** write operation
- If the write doesn't complete within `writeWait` (10 seconds), it fails with an error
- Always set deadlines before writing — without them, a stalled connection blocks the goroutine forever

Message Size Limits

```
c.conn.SetReadLimit(maxMessageSize)
```

- Sets the maximum message size the server will accept (e.g., 512 bytes)
- If a client sends a message larger than this, `ReadMessage` returns an error and the connection closes
- Prevents clients from sending enormous payloads that consume server memory
- Choose a limit appropriate for your application: 512 bytes for chat, 4 KB for rich content, etc.

Reconnection Strategy (JavaScript Client)

```
function connect() {
    ws = new WebSocket('ws://localhost:8080/ws');
    ws.onclose = function() {
        // Reconnect after a delay
        setTimeout(connect, 3000);
    };
}
```

A more robust approach uses **exponential backoff**:

```
let reconnectDelay = 1000; // Start with 1 second
const maxDelay = 30000;    // Cap at 30 seconds

function connect() {
    ws = new WebSocket('ws://localhost:8080/ws');

    ws.onopen = function() {
        reconnectDelay = 1000; // Reset delay on successful connection
    };

    ws.onclose = function() {
        console.log('Reconnecting in', reconnectDelay, 'ms');
        setTimeout(connect, reconnectDelay);
        reconnectDelay = Math.min(reconnectDelay * 2, maxDelay);
    };
}
```

This prevents hammering the server with reconnection attempts when it's down.

Sending Error Messages to Clients

```
func sendError(conn *websocket.Conn, message string) {
    errMsg := Message{
        Type:    "error",
        Content: message,
    }
    data, _ := json.Marshal(errMsg)
    conn.WriteMessage(websocket.TextMessage, data)
}
```

- Use a consistent error message format so clients can detect and display errors
- The `type: "error"` field lets the client distinguish errors from normal messages

Key Connection Management Concepts

1. **Deadlines prevent hanging**: always set write deadlines before sending
2. **Read limits prevent abuse**: cap message sizes to protect server memory
3. **Reconnection with backoff**: clients should automatically reconnect with increasing delays
4. **Structured errors**: send error messages as JSON with `type: "error"` so clients can handle them

---

Part 7: Testing WebSocket Services

📖 Sample: Testing WebSocket Without Writing Client Code

Method 1: Browser DevTools

The easiest way to test WebSocket is with your browser's developer tools:

1. Open `http://localhost:8080` in Chrome/Firefox
2. Open DevTools (F12) → **Network** tab
3. Filter by **WS** (WebSocket)
4. Click on the WebSocket connection to see frames
5. You'll see each message sent and received with timestamps

This is the WebSocket equivalent of watching HTTP requests in the Network tab.

Method 2: `websocat` (CLI Tool)

`websocat` is a command-line WebSocket client — the WebSocket equivalent of `curl` or `grpcurl`.

Install:

```
# Ubuntu/Debian (download from GitHub releases)
# Or build from source with Rust
cargo install --features=ssl websocat

# macOS
brew install websocat
```

Usage:

```
# Connect and send/receive messages interactively
websocat ws://localhost:8080/ws

# Type a message, press Enter — see the response
Hello
← Hello

# Send a JSON message
{"type":"chat","sender":"CLI","content":"Hello from terminal"}
```

Non-interactive (one-shot send):

```
echo '{"type":"chat","sender":"test","content":"automated"}' | websocat ws://localh
```

Method 3: Go Test Client

For automated testing, write a Go test:

```
func TestEcho(t *testing.T) {
    // Start server (or assume it's running)
    conn, _, err := websocket.DefaultDialer.Dial("ws://localhost:8080/ws", nil)
    if err != nil {
        t.Fatalf("dial failed: %v", err)
    }
    defer conn.Close()

    // Send a message
    msg := []byte("hello")
    err = conn.WriteMessage(websocket.TextMessage, msg)
    if err != nil {
        t.Fatalf("write failed: %v", err)
    }

    // Read the echo
    _, response, err := conn.ReadMessage()
    if err != nil {
        t.Fatalf("read failed: %v", err)
    }

    if string(response) != "hello" {
        t.Fatalf("expected 'hello', got '%s'", response)
    }
}
```

Method 4: Verify the Upgrade Handshake with `curl`

You can use `curl` to inspect the initial HTTP upgrade request (though `curl` can't continue the WebSocket conversation):

```
curl -i \
  -H "Upgrade: websocket" \
  -H "Connection: Upgrade" \
  -H "Sec-WebSocket-Key: dGVzdA==" \
  -H "Sec-WebSocket-Version: 13" \
  http://localhost:8080/ws
```

Expected response:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: ...
```

This confirms the server is properly handling the WebSocket upgrade.

Common Mistakes to Avoid

❌ Mistake 1: No Read Loop — Connection Closes Immediately

```go
// WRONG — handler returns, connection is garbage collected
func handler(w http.ResponseWriter, r *http.Request) {
    conn, _ := upgrader.Upgrade(w, r, nil)
    conn.WriteMessage(websocket.TextMessage, []byte("hello"))
    // Function returns → connection drops
}
```

✅ Solution: Always Run a Read Loop

```go
// CORRECT — read loop keeps the connection alive
func handler(w http.ResponseWriter, r *http.Request) {
    conn, _ := upgrader.Upgrade(w, r, nil)
    defer conn.Close()
    for {
        _, _, err := conn.ReadMessage()
        if err != nil { break }
    }
}
```

❌ Mistake 2: Concurrent Writes — Panic or Corrupted Frames

```go
// WRONG — two goroutines writing to the same connection
go func() { conn.WriteMessage(websocket.TextMessage, []byte("from goroutine 1")) }()
go func() { conn.WriteMessage(websocket.TextMessage, []byte("from goroutine 2")) }()
```

✅ Solution: Use a Channel + Dedicated Writer

```go
// CORRECT — only one goroutine writes
send := make(chan []byte, 256)
go func() {
    for msg := range send {
        conn.WriteMessage(websocket.TextMessage, msg)
    }
}()

// Other goroutines send via the channel
send <- []byte("safe message 1")
send <- []byte("safe message 2")
```

❌ Mistake 3: No Ping/Pong — Dead Connections Go Undetected

```go
// WRONG — if the client's network drops, you'll never know
for {
    _, msg, err := conn.ReadMessage()
    if err != nil { break }
    // This loop blocks forever on a silently dead connection
}
```

✅ Solution: Set Deadlines and Use Ping/Pong

```go
// CORRECT — readPump with pong handler
conn.SetReadDeadline(time.Now().Add(60 * time.Second))
conn.SetPongHandler(func(string) error {
    conn.SetReadDeadline(time.Now().Add(60 * time.Second))
    return nil
})

// writePump sends periodic pings
ticker := time.NewTicker(54 * time.Second)
for {
    select {
    case <-ticker.C:
        conn.WriteMessage(websocket.PingMessage, nil)
    // ...
    }
}
```

❌ Mistake 4: Not Cleaning Up Disconnected Clients

```go
// WRONG — client disconnects but stays in the map
// Hub tries to send to dead clients, causing errors
```

✅ Solution: Unregister in `readPump` 's `defer`

```go
func (c *Client) readPump() {
    defer func() {
        c.hub.unregister <- c  // Remove from Hub
        c.conn.Close()         // Close connection
    }()
    // ... read loop ...
}
```

❌ Mistake 5: Abrupt Close Without Close Frame

```go
// WRONG — client sees error code 1006 (abnormal closure)
conn.Close()
```

✅ Solution: Send a Close Frame First

```go
// CORRECT — client sees clean close with code 1000
conn.WriteMessage(
    websocket.CloseMessage,
    websocket.FormatCloseMessage(websocket.CloseNormalClosure, "goodbye"),
)
conn.Close()
```

❌ Mistake 6: Using `localhost` in Docker

```go
// WRONG inside a container
conn, _, _ := websocket.DefaultDialer.Dial("ws://localhost:8080/ws", nil)
```

✅ Solution: Use Docker Service Name

```go
// CORRECT — use the service name from docker-compose.yml
conn, _, _ := websocket.DefaultDialer.Dial("ws://chat-server:8080/ws", nil)
```

❌ Mistake 7: Ignoring `CheckOrigin` in Production

```go
// WRONG for production — allows any website to connect
CheckOrigin: func(r *http.Request) bool { return true }
```

✅ Solution: Validate the Origin

```go
// CORRECT for production
CheckOrigin: func(r *http.Request) bool {
    origin := r.Header.Get("Origin")
    return origin == "https://yourdomain.com"
}
```

❌ Mistake 8: Sending Raw Strings Instead of JSON

```go
// WRONG — hard to parse, no metadata, no routing
conn.WriteMessage(websocket.TextMessage, []byte("Alice: Hello!"))
```

✅ Solution: Use Structured JSON Messages

```go
// CORRECT — structured, parseable, extensible
msg := Message{Type: "chat", Sender: "Alice", Content: "Hello!"}
data, _ := json.Marshal(msg)
conn.WriteMessage(websocket.TextMessage, data)
```

Quick Reference

Server Boilerplate

```go
package main

import (
    "log"
    "net/http"
    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
    ReadBufferSize:  1024,
    WriteBufferSize: 1024,
    CheckOrigin: func(r *http.Request) bool { return true },
}

func handleWebSocket(w http.ResponseWriter, r *http.Request) {
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil { log.Println(err); return }
    defer conn.Close()

    for {
        msgType, msg, err := conn.ReadMessage()
        if err != nil { break }
        conn.WriteMessage(msgType, msg)
    }
}

func main() {
    http.HandleFunc("/ws", handleWebSocket)
    http.Handle("/", http.FileServer(http.Dir("static")))
    log.Println("Server on :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Hub + Client Boilerplate

```go
// Hub
type Hub struct {
    clients    map[*Client]bool
    broadcast  chan []byte
    register   chan *Client
    unregister chan *Client
}

// Client
type Client struct {
    hub  *Hub
    conn *websocket.Conn
    send chan []byte
}

// Launch
hub := newHub()
go hub.run()
```

Go Client Boilerplate

```go
conn, _, err := websocket.DefaultDialer.Dial("ws://localhost:8080/ws", nil)
if err != nil { log.Fatal(err) }
defer conn.Close()

// Read in goroutine
go func() {
    for {
        _, msg, err := conn.ReadMessage()
        if err != nil { return }
        fmt.Printf("← %s\n", msg)
    }
}()

// Write from main
conn.WriteMessage(websocket.TextMessage, []byte("hello"))
```

JavaScript Client Boilerplate

```javascript
const ws = new WebSocket('ws://localhost:8080/ws');
ws.onopen = () => console.log('connected');
ws.onmessage = (e) => console.log('received:', e.data);
ws.onclose = () => console.log('disconnected');
ws.send(JSON.stringify({type: 'chat', sender: 'user', content: 'hello'}));
```

JSON Message Struct

```go
type Message struct {
    Type      string `json:"type"`
    Sender    string `json:"sender"`
    Content   string `json:"content"`
    Timestamp string `json:"timestamp,omitempty"`
}

// Parse incoming
var msg Message
json.Unmarshal(rawBytes, &msg)

// Route
switch msg.Type {
case "chat":   // broadcast
case "join":   // notify others
case "leave":  // notify others
case "dm":     // send to specific user
case "error":  // send error to sender
}

// Build outgoing
data, _ := json.Marshal(msg)
conn.WriteMessage(websocket.TextMessage, data)
```

Key `gorilla/websocket` Constants

```go
// Message types
websocket.TextMessage   = 1   // UTF-8 text
websocket.BinaryMessage = 2   // Raw bytes
websocket.CloseMessage  = 8   // Close frame
websocket.PingMessage   = 9   // Ping frame
websocket.PongMessage   = 10  // Pong frame

// Close codes
websocket.CloseNormalClosure    = 1000  // Clean shutdown
websocket.CloseGoingAway        = 1001  // Browser tab closed
websocket.CloseProtocolError    = 1002  // Protocol violation
websocket.CloseAbnormalClosure  = 1006  // No close frame received
```

Dependencies Summary

```bash
# Install gorilla/websocket
go get github.com/gorilla/websocket

# Optional: install websocat for CLI testing
cargo install --features=ssl websocat  # or: brew install websocat
```

Best Practices Summary

✅ DO:

1. **Always run a read loop** — it keeps the connection alive and processes control frames
2. **Use the Hub pattern** for multi-client communication — channels, not mutexes
3. **Use readPump/writePump** — one goroutine per direction per connection
4. **Send structured JSON** with a `type` field for message routing
5. **Implement ping/pong** to detect dead connections
6. **Set read limits** to prevent oversized messages from consuming memory
7. **Set write deadlines** to prevent stalled writes from blocking goroutines
8. **Clean up in `defer`** — unregister from Hub and close connection
9. **Use buffered `send` channels** to absorb message bursts
10. **Validate input** — check JSON structure, required fields, message size

❌ DON'T:

1. **Don't write from multiple goroutines** — use a channel + single writer
2. **Don't forget `defer conn.Close()`** — leaked connections consume resources
3. **Don't skip `CheckOrigin`** in production — it's the WebSocket equivalent of CORS
4. **Don't send raw strings** — use structured JSON for maintainability
5. **Don't ignore disconnections** — always unregister dead clients from the Hub
6. **Don't use `ws://` in production** — use `wss://` (TLS) for security
7. **Don't use `localhost` in Docker** — use the Docker service name
8. **Don't close without a Close frame** — send `CloseMessage` for clean disconnection
9. **Don't block the read loop** — process messages quickly or offload to a goroutine
10. **Don't assume message order** — in broadcast, different clients may receive messages in slightly different order

---

Next Steps

Now that you've studied the sample code:

1. ✅ Make sure you understand each sample
2. ✅ Run the samples on your machine
3. ✅ Test with browser DevTools and Go client
4. ✅ Experiment with modifying the message protocol
5. ➡️ **Now attempt the tasks in the WebSocket Lab Tasks file**

Good luck! 🚀

End of WebSocket Lab Sample Code Guide